LAB 7

Fuzz Testing for Input Validation Bugs

ATM withdrawal application · Python · Hypothesis · pytest

Student	Anvi Srinivas
SRN	PES1UG24CS076
Class	5B
Application	Lab 6 ATM application, used as processor.py

This report records the input-validation work carried out on the ATM application. The original implementation was exercised, the three functions were hardened, and Hypothesis tests were used to check valid and malformed inputs.

What is included: the original and corrected execution evidence, the final successful pytest run, a summary of the issues addressed, and the complete corrected application and test source code.

1. What was tested

The application contains three functions. **withdraw()** checks account details, balances, withdrawal amounts and bank fees. **pin_check()** checks a PIN and the number of attempts. **note_breakup()** calculates a breakdown of an amount into ■500, ■200 and ■100 notes.

The tests use Hypothesis to generate a mixture of ordinary values and edge cases, including None, booleans, floats, NaN, strings, lists, negative numbers and out-of-range values. The intended property is that malformed input should be rejected cleanly rather than causing an unexpected exception or producing an invalid result.

2. Issues identified and changes made

Function	Input issue	Correction
withdraw()	Invalid account types, unhashable account values, None/NaN/negative balances, non-integer amounts and non-boolean bank flags.	Validate every argument before applying the withdrawal rules. Reject invalid values with ValueError.
pin_check()	Missing or malformed PINs, booleans used as integers, invalid stored PINs and negative/non-integer attempt counts.	Require integer PINs from 0 to 9999 and a non-negative integer attempt count.
note_breakup()	None, booleans, floats, negative values and amounts not divisible by ■100.	Require a non-negative integer amount divisible by ■100 before calculating notes.

The corrected code keeps the original application rules: savings/current account limits, the external-bank fee, the three-attempt card lock and the note calculation. The main change is that invalid inputs are checked before the business logic runs.

3. Execution evidence

Screenshot 1 — Original application

The original program demonstrates why input validation is needed. Some malformed inputs caused TypeError exceptions; other inputs produced unsafe or misleading results, such as NaN being returned, matching empty PIN values, or a negative note count.

```
C:\Users\srini\Documents\SE\Lab7>prompt AnviPES1UG24CS076$G

AnviPES1UG24CS076>cd original_buggy

AnviPES1UG24CS076>python processor.py
=== withdraw ===
withdraw(5000, 1000, 'savings', True)               -> 4000
withdraw(5000, 1000, 'savings', False)              -> 3980
withdraw(5000, 1000, 'salary', True)                -> ValueError: unknown account
withdraw(None, 1000, 'savings', True)               -> TypeError: '>' not supported be
tween instances of 'int' and 'NoneType'
withdraw(5000, None, 'savings', True)               -> TypeError: '<=' not supported b
etween instances of 'NoneType' and 'int'
withdraw(5000, 1000, ['savings'], True)             -> TypeError: cannot use 'list' as
 a dict key (unhashable type: 'list')
withdraw(nan, 1000, 'savings', True)                -> nan
withdraw(-500, 100, 'savings', True)                -> 'insufficient funds'
withdraw(5000, 1000.0, 'savings', True)             -> 4000.0
withdraw(5000, 1000, 'savings', None)               -> 3980

=== pin_check ===
pin_check(1234, 1234, 0)                            -> 'ok'
pin_check(1111, 1234, 2)                            -> 'card blocked'
pin_check(None, None, 0)                            -> 'ok'
pin_check(True, 1, 0)                               -> 'ok'
pin_check(1234, 1234, None)                         -> TypeError: '>=' not supported b
etween instances of 'NoneType' and 'int'
pin_check(1111, 1234, -5)                           -> 'retry'

=== note_breakup ===
note_breakup(1200)                                  -> (2, 1, 0)
note_breakup(150)                                   -> ValueError: invalid amount
note_breakup(None)                                  -> TypeError: unsupported operand
type(s) for %: 'NoneType' and 'int'
note_breakup(-100)                                  -> (-1, 2, 0)
note_breakup(100.0)                                 -> (0.0, 0, 1.0)
note_breakup(False)                                 -> (0, 0, 0)

AnviPES1UG24CS076>
```

Screenshot 2 — Fuzz-test failure evidence

The supplied screenshots show the original program's incorrect outputs, but a separate terminal screenshot of pytest/Hypothesis failing against the original version was not included. The failure examples are therefore described in the findings section rather than represented by a fabricated test-run image. If the lab evaluator requires the actual failing pytest screen, run the tests from the original_buggy folder and add that screenshot.

Command to capture the original failing run:

```
cd original_buggy
python -m pytest -q test_processor.py
```

Screenshot 3 — Corrected application

The corrected application now rejects malformed inputs with clear ValueError messages. Valid calls continue to return the expected withdrawal result, PIN status or note breakdown.

```
C:\Users\Anvi\Documents\SE\Lab7> python processor.py

=== withdraw ===
withdraw(5000, 1000, 'savings', True)               -> 4000
withdraw(5000, 1000, 'savings', False)              -> 3980
withdraw(5000, 1000, 'salary', True)                -> ValueError: unknown account
withdraw(None, 1000, 'savings', True)               -> ValueError: invalid balance
withdraw(5000, None, 'savings', True)               -> ValueError: invalid amount
withdraw(5000, 1000, ['savings'], True)             -> ValueError: unknown account
withdraw(nan, 1000, 'savings', True)                -> ValueError: invalid balance
withdraw(-500, 100, 'savings', True)                -> ValueError: invalid balance
withdraw(5000, 1000.0, 'savings', True)             -> ValueError: invalid amount
withdraw(5000, 1000, 'savings', None)               -> ValueError: invalid own_bank flag

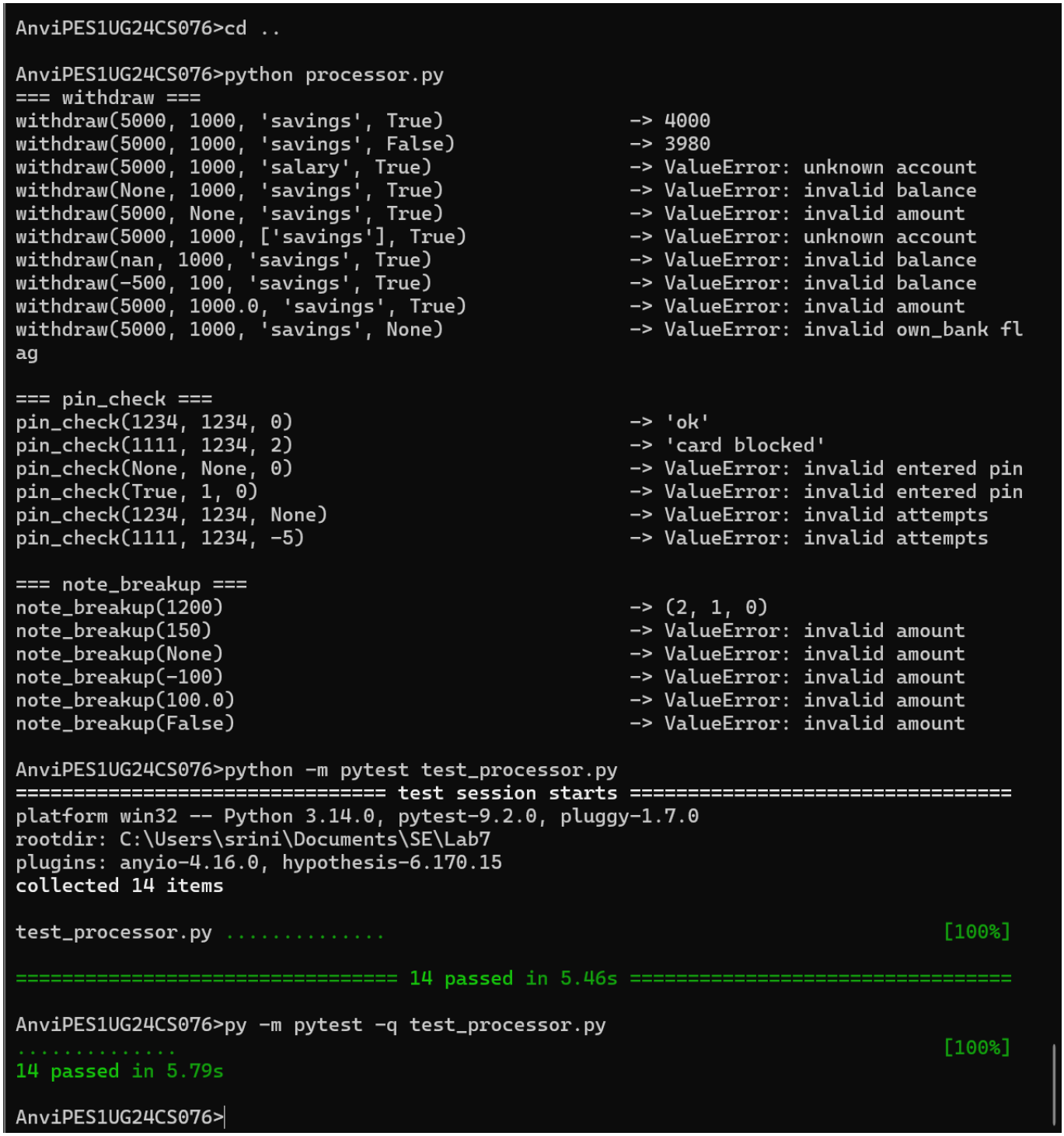
=== pin_check ===
pin_check(1234, 1234, 0)                            -> 'ok'
pin_check(1111, 1234, 2)                            -> 'card blocked'
pin_check(None, None, 0)                            -> ValueError: invalid entered pin
pin_check(True, 1, 0)                               -> ValueError: invalid entered pin
pin_check(1234, 1234, None)                         -> ValueError: invalid attempts
pin_check(1111, 1234, -5)                           -> ValueError: invalid attempts

=== note_breakup ===
note_breakup(1200)                                  -> (2, 1, 0)
note_breakup(150)                                   -> ValueError: invalid amount
note_breakup(None)                                  -> ValueError: invalid amount
note_breakup(-100)                                  -> ValueError: invalid amount
note_breakup(100.0)                                 -> ValueError: invalid amount
note_breakup(False)                                 -> ValueError: invalid amount

AnviPES1UG24CS076>
```

Screenshot 4 — Final test execution

The final screenshot shows the corrected application output followed by the pytest run. All 14 collected tests passed. The screenshot includes a normal pytest run (14 passed in 5.46s) and a quiet rerun (14 passed in 5.79s).



4. Corrected application source — processor.py

```
"""
Lab 7 - FIXED version of the ATM application (Lab 6: atm_withdrawal.py).

Hardening done after fuzz testing with Hypothesis:
  * every argument is type- and range-checked before it is used
  * every invalid input raises ValueError (never TypeError / silent garbage)
  * bool is NOT accepted where a number is expected (True == 1 in Python)
  * float / NaN / inf are rejected where whole numbers are required
Business rules (limits, fees, 3-attempt PIN lock, note split) are unchanged,
so all the Lab 6 tests still pass.
"""

DAILY_LIMIT = {"savings": 25000, "current": 50000}
MAX_PIN = 9999


def _is_int(value):
    """True only for real integers (bool is a subclass of int, so exclude it)."""
    return isinstance(value, int) and not isinstance(value, bool)


def _is_pin(value):
    return _is_int(value) and 0 <= value <= MAX_PIN


def withdraw(balance, amount, account_type, own_bank):
    # --- input validation ------------------------------------------------
    if not isinstance(account_type, str) or account_type not in DAILY_LIMIT:
        raise ValueError("unknown account")
    if not _is_int(amount) or amount <= 0 or amount % 100 != 0:
        raise ValueError("invalid amount")
    if not _is_int(balance) or balance < 0:
        raise ValueError("invalid balance")
    if not isinstance(own_bank, bool):
        raise ValueError("invalid own_bank flag")

    # --- business logic (unchanged) --------------------------------------
    if amount > DAILY_LIMIT[account_type]:
        return "limit exceeded"
    fee = 0
    if not own_bank:
        fee = 20
        if amount > 10000:
            fee += 10
    if amount + fee > balance:
        return "insufficient funds"
    return balance - amount - fee


def pin_check(entered, actual, attempts):
    # --- input validation ------------------------------------------------
    if not _is_int(attempts) or attempts < 0:
        raise ValueError("invalid attempts")
    if not _is_pin(entered):
        raise ValueError("invalid entered pin")
    if not _is_pin(actual):
        raise ValueError("invalid actual pin")

    # --- business logic (unchanged) --------------------------------------
    if attempts >= 3:
        return "card blocked"
    if entered == actual:
        return "ok"
    if attempts == 2:
        return "card blocked"
    return "retry"


def note_breakup(amount):
    # --- input validation ------------------------------------------------
    if not _is_int(amount) or amount < 0 or amount % 100 != 0:
        raise ValueError("invalid amount")

    # --- business logic (unchanged) --------------------------------------
    notes_500 = amount // 500
    remaining = amount % 500
    if remaining >= 200:
        notes_200 = remaining // 200
        remaining = remaining % 200
    else:
        notes_200 = 0
    return notes_500, notes_200, remaining // 100


# ---------------------------------------------------------------------------
# Demo: identical calls to the original version, so the outputs can be compared
# ---------------------------------------------------------------------------
def _try(label, func, *args):
    call = f"{label}({', '.join(repr(a) for a in args)})"
    try:
        result = func(*args)
        print(f"{call:<52} -> {result!r}")
    except Exception as exc:  # noqa: BLE001 - demo only
        print(f"{call:<52} -> {type(exc).__name__}: {exc}")
if __name__ == "__main__":
```

```
print("=== withdraw ===")
_try("withdraw", withdraw, 5000, 1000, "savings", True)
_try("withdraw", withdraw, 5000, 1000, "savings", False)
_try("withdraw", withdraw, 5000, 1000, "salary", True)
_try("withdraw", withdraw, None, 1000, "savings", True)
_try("withdraw", withdraw, 5000, None, "savings", True)
_try("withdraw", withdraw, 5000, 1000, ["savings"], True)
_try("withdraw", withdraw, float("nan"), 1000, "savings", True)
_try("withdraw", withdraw, -500, 100, "savings", True)
_try("withdraw", withdraw, 5000, 1000.0, "savings", True)
_try("withdraw", withdraw, 5000, 1000, "savings", None)

print("\n=== pin_check ===")
_try("pin_check", pin_check, 1234, 1234, 0)
_try("pin_check", pin_check, 1111, 1234, 2)
_try("pin_check", pin_check, None, None, 0)
_try("pin_check", pin_check, True, 1, 0)
_try("pin_check", pin_check, 1234, 1234, None)
_try("pin_check", pin_check, 1111, 1234, -5)

print("\n=== note_breakup ===")
_try("note_breakup", note_breakup, 1200)
_try("note_breakup", note_breakup, 150)
_try("note_breakup", note_breakup, None)
_try("note_breakup", note_breakup, -100)
_try("note_breakup", note_breakup, 100.0)
_try("note_breakup", note_breakup, False)
```

5. Hypothesis test source — test_processor.py

```
"""
Lab 7 - Hypothesis fuzz tests for processor.py (withdraw, pin_check, note_breakup).

Three kinds of properties are checked for every function:
  1. NO-CRASH   : for ANY input the function either returns a well-formed result
                  or raises ValueError. Any other exception (TypeError, ...) or a
                  malformed result (e.g. float nan) is a bug.
  2. REJECTION  : clearly invalid input must raise ValueError.
  3. INVARIANTS : for valid input the result must satisfy the business rules.

Run:  pytest test_processor.py      (or: python -m pytest test_processor.py)
"""
import pytest
from hypothesis import given, settings, assume, strategies as st

from processor import withdraw, pin_check, note_breakup, DAILY_LIMIT

SETTINGS = settings(max_examples=300, deadline=None)

# ---------------------------------------------------------------------------
# Strategies
# ---------------------------------------------------------------------------
# "Anything": every kind of value a caller could wrongly pass in.
anything = st.one_of(
    st.none(),
    st.booleans(),
    st.integers(),
    st.floats(),                                # includes nan, inf, -inf
    st.text(),
    st.binary(),
    st.lists(st.integers()),                    # unhashable
    st.dictionaries(st.text(), st.integers()),  # unhashable
)

# Valid values
valid_account = st.sampled_from(sorted(DAILY_LIMIT))
valid_amount = st.integers(min_value=1, max_value=600).map(lambda k: k * 100)
valid_balance = st.integers(min_value=0, max_value=10_000_000)
valid_pin = st.integers(min_value=0, max_value=9999)
valid_attempts = st.integers(min_value=0, max_value=10)
valid_notes_amount = st.integers(min_value=0, max_value=10_000).map(lambda k: k * 100)


def mixed(valid):
    """Mostly-valid values with the odd garbage value mixed in (reaches deeper code)."""
    return st.one_of(valid, valid, anything)


def is_real_int(x):
    return isinstance(x, int) and not isinstance(x, bool)


def check_only_value_error(func, *args):
    """Call func; allow ValueError, fail on any other exception. Return result or None."""
    try:
        return func(*args)
    except ValueError:
        return None
    except Exception as exc:  # noqa: BLE001
        pytest.fail(f"{func.__name__}{args!r} crashed with {type(exc).__name__}: {exc}")


# ===========================================================================
# withdraw
# ===========================================================================
@SETTINGS
@given(balance=mixed(valid_balance), amount=mixed(valid_amount),
       account=mixed(valid_account), own_bank=mixed(st.booleans()))
def test_withdraw_never_crashes(balance, amount, account, own_bank):
    """Any input -> ValueError or a well-formed result (never TypeError, never nan)."""
    result = check_only_value_error(withdraw, balance, amount, account, own_bank)
    if result is not None:
        assert is_real_int(result) or result in ("limit exceeded", "insufficient funds"), \
            f"malformed result {result!r}"


@SETTINGS
@given(account=anything, balance=valid_balance, amount=valid_amount, own_bank=st.booleans())
def test_withdraw_rejects_bad_account_type(account, balance, amount, own_bank):
    assume(not (isinstance(account, str) and account in DAILY_LIMIT))
    with pytest.raises(ValueError):
        withdraw(balance, amount, account, own_bank)


@SETTINGS
@given(
    amount=st.one_of(
        st.integers().filter(lambda x: x <= 0 or x % 100 != 0),
        st.floats(), st.text(), st.none(), st.booleans(), st.lists(st.integers())),
    balance=valid_balance, account=valid_account, own_bank=st.booleans())
def test_withdraw_rejects_bad_amount(amount, balance, account, own_bank):
    """Zero, negative, non-multiple of 100, floats (incl. 100.0 / nan / inf), None, bool, str."""
    with pytest.raises(ValueError):
        withdraw(balance, amount, account, own_bank)
@SETTINGS
```

```
@given(
    balance=st.one_of(st.integers(max_value=-1), st.floats(), st.text(),
                      st.none(), st.booleans(), st.lists(st.integers())),
    amount=valid_amount, account=valid_account, own_bank=st.booleans())
def test_withdraw_rejects_bad_balance(balance, amount, account, own_bank):
    """Negative, float/nan, None, bool, str balances must be rejected."""
    with pytest.raises(ValueError):
        withdraw(balance, amount, account, own_bank)


@SETTINGS
@given(own_bank=st.one_of(st.none(), st.integers(), st.floats(), st.text(),
                          st.lists(st.integers())),
       balance=valid_balance, amount=valid_amount, account=valid_account)
def test_withdraw_rejects_non_bool_own_bank(own_bank, balance, amount, account):
    with pytest.raises(ValueError):
        withdraw(balance, amount, account, own_bank)


@SETTINGS
@given(balance=valid_balance, amount=valid_amount, account=valid_account, own_bank=st.booleans())
def test_withdraw_invariants(balance, amount, account, own_bank):
    result = withdraw(balance, amount, account, own_bank)
    if amount > DAILY_LIMIT[account]:
        assert result == "limit exceeded"
    elif isinstance(result, str):
        assert result == "insufficient funds"
        required_fee = 0 if own_bank else (30 if amount > 10000 else 20)
        assert balance < amount + required_fee
    else:
        assert is_real_int(result)
        assert 0 <= result <= balance - amount               # money never created
        fee = balance - amount - result
        if own_bank:
            assert fee == 0                                  # own bank is free
        else:
            assert fee == (30 if amount > 10000 else 20)     # other-bank fee tiers


# ============================================================================
# pin_check
# ============================================================================
@SETTINGS
@given(entered=mixed(valid_pin), actual=mixed(valid_pin), attempts=mixed(valid_attempts))
def test_pin_check_never_crashes(entered, actual, attempts):
    result = check_only_value_error(pin_check, entered, actual, attempts)
    if result is not None:
        assert result in ("ok", "retry", "card blocked")


@SETTINGS
@given(entered=st.one_of(st.none(), st.booleans(), st.floats(), st.text(),
                         st.integers(max_value=-1), st.integers(min_value=10_000),
                         st.lists(st.integers())),
       actual=valid_pin, attempts=valid_attempts)
def test_pin_check_rejects_bad_entered_pin(entered, actual, attempts):
    with pytest.raises(ValueError):
        pin_check(entered, actual, attempts)


@SETTINGS
@given(actual=st.one_of(st.none(), st.booleans(), st.floats(), st.text(),
                        st.integers(max_value=-1), st.integers(min_value=10_000),
                        st.lists(st.integers())),
       entered=valid_pin, attempts=valid_attempts)
def test_pin_check_rejects_bad_stored_pin(actual, entered, attempts):
    """Regression: pin_check(None, None, 0) used to return 'ok' (empty PIN matched)."""
    with pytest.raises(ValueError):
        pin_check(entered, actual, attempts)


@SETTINGS
@given(attempts=st.one_of(st.integers(max_value=-1), st.none(), st.floats(),
                          st.text(), st.booleans(), st.lists(st.integers())),
       entered=valid_pin, actual=valid_pin)
def test_pin_check_rejects_bad_attempts(attempts, entered, actual):
    with pytest.raises(ValueError):
        pin_check(entered, actual, attempts)


@SETTINGS
@given(entered=valid_pin, actual=valid_pin, attempts=valid_attempts)
def test_pin_check_invariants(entered, actual, attempts):
    result = pin_check(entered, actual, attempts)
    if attempts >= 3:
        assert result == "card blocked"          # blocked card never opens, even with right PIN
    elif entered == actual:
        assert result == "ok"
    elif attempts == 2:
        assert result == "card blocked"          # third wrong try blocks the card
    else:
        assert result == "retry"


# ============================================================================
# note_breakup
# ============================================================================
@SETTINGS
@given(amount=mixed(valid_notes_amount))
```

```
def test_note_breakup_never_crashes(amount):
    result = check_only_value_error(note_breakup, amount)
    if result is not None:
        assert isinstance(result, tuple) and len(result) == 3
        assert all(is_real_int(n) and n >= 0 for n in result), f"bad note counts {result!r}"


@SETTINGS
@given(amount=st.one_of(
    st.integers().filter(lambda x: x < 0 or x % 100 != 0),
    st.floats(), st.text(), st.none(), st.booleans(), st.lists(st.integers())))
def test_note_breakup_rejects_bad_amount(amount):
    """Negative amounts, non-multiples of 100, floats (incl. 100.0), None, bool, str."""
    with pytest.raises(ValueError):
        note_breakup(amount)


@SETTINGS
@given(amount=valid_notes_amount)
def test_note_breakup_invariants(amount):
    n500, n200, n100 = note_breakup(amount)
    assert n500 >= 0 and n200 >= 0 and n100 >= 0
    assert 500 * n500 + 200 * n200 + 100 * n100 == amount   # totals add up exactly
    assert n200 <= 2 and n100 <= 1                           # greedy split: remainder < 500
```